

# K8s-08

## 1. Create and test a Kubernetes Pod with an Empty Dir Volume

- Create an Empty Dir Volume yaml file

```
root@Master:~/k8s-08# vi emptydir-pod.yaml
root@Master:~/k8s-08#
```

```
apiVersion: v1
kind: Pod
metadata:
  name: emptydir-pod
spec:
  containers:
  - name: nginx
    image: nginx

    volumeMounts:
    - name: cache-volume
      mountPath: /usr/share/nginx/html

  volumes:
  - name: cache-volume
    emptyDir: {}
```

- Apply and verify it

```
root@Master:~/k8s-08# kubectl apply -f emptydir-pod.yaml
pod/emptydir-pod created
root@Master:~/k8s-08# kubectl get pod emptydir-pod
NAME          READY   STATUS    RESTARTS   AGE
emptydir-pod  1/1     Running   0           6s
root@Master:~/k8s-08#
```

- Create a file inside the volume and exit

```
root@Master:~/k8s-08# kubectl exec -it emptydir-pod -- sh
# echo "Hello EmptyDir" > /usr/share/nginx/html/index.html

cat /usr/share/nginx/html/index.html# #
Hello EmptyDir
# exit
```

- Now verify from outside it

```
root@Master:~/k8s-08# kubectl exec emptydir-pod -- cat /usr/share/nginx/html/index.html
Hello EmptyDir
root@Master:~/k8s-08#
```

## 2. Configure a Host Path Volume in Kubernetes and Validate Data Persistence

- Create a directory on the node and verify it

```
root@Master:~/k8s-08# mkdir -p /data/hostpath
echo "HostPath Volume Test" > /data/hostpath/index.html
root@Master:~/k8s-08# cat /data/hostpath/index.html
HostPath Volume Test
root@Master:~/k8s-08#
```

- Now Create host path-pod. Yaml

```
root@Master:~/k8s-08# vi hostpath-pod.yaml
root@Master:~/k8s-08#
```

```
apiVersion: v1
kind: Pod
metadata:
  name: hostpath-pod
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: host-volume
      mountPath: /usr/share/nginx/html
  volumes:
  - name: host-volume
    hostPath:
      path: /data/hostpath
      type: Directory
```

- Apply the yaml file and verify pod

```
root@Master:~/k8s-08# kubectl apply -f hostpath-pod.yaml
pod/hostpath-pod created
root@Master:~/k8s-08# kubectl get pod hostpath-pod
NAME          READY   STATUS    RESTARTS   AGE
hostpath-pod  1/1     Running   0           7s
root@Master:~/k8s-08#
```

- Verify the mounted file

```
root@Master:~/k8s-08# kubectl exec hostpath-pod -- cat /usr/share/nginx/html/index.html
HostPath Volume Test
root@Master:~/k8s-08#
```

- Test data persistence (Create another file inside the pod) and exit

```
root@Master:~/k8s-08# kubectl exec -it hostpath-pod -- sh
# echo "Persistent Data" > /usr/share/nginx/html/test.txt
# exit
root@Master:~/k8s-08#
```

- Verify from the node

```
root@Master:~/k8s-08# cat /data/hostpath/test.txt
Persistent Data
root@Master:~/k8s-08#
```

### 3. Deploy an Amazon EBS Volume Using Persistent Volume and Persistent Volume Claim (PVC)

- Create pv.yaml file

```
root@Master:~/k8s-08# mkdir -p /data/pv
root@Master:~/k8s-08# vi pv.yaml
root@Master:~/k8s-08#
```

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: host-pv
spec:
  capacity:
    storage: 2Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  hostPath:
    path: /data/pv
```

- Apply and verify it

```
root@Master:~/k8s-08# kubectl apply -f pv.yaml
persistentvolume/host-pv created
root@Master:~/k8s-08# kubectl get pv
NAME          CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS    CLAIM  STORAGECLASS  V
host-pv       2Gi      RWO           Retain          Available
root@Master:~/k8s-08#
```

- Create Pvc.yaml file

```
root@Master:~/k8s-08#
root@Master:~/k8s-08# vi pvc.yaml
root@Master:~/k8s-08#
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: host-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

- Apply and verify pvc and pv also

```
root@Master:~/k8s-08# kubectl apply -f pvc.yaml
persistentvolumeclaim/host-pvc created
root@Master:~/k8s-08# kubectl get pvc
NAME          STATUS  VOLUME  CAPACITY  ACCESS MODES  STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
host-pvc      Bound   host-pv  2Gi      RWO           <unset>        <unset>                 7s
root@Master:~/k8s-08# kubectl get pv
NAME          CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS    CLAIM  STORAGECLASS  VOLUMEATTRIBUTESCLASS
host-pv       2Gi      RWO           Retain          Bound     default/host-pvc  <unset>
root@Master:~/k8s-08#
```

- Create pv-pod.yaml file

```
root@Master:~/k8s-08#
root@Master:~/k8s-08# vi pv-pod.yaml
root@Master:~/k8s-08#
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pv-pod
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: data
      mountPath: /usr/share/nginx/html

  volumes:
  - name: data
    persistentVolumeClaim:
      claimName: host-pvc
```

- Apply and verify it

```
root@Master:~/k8s-08# kubectl apply -f pv-pod.yaml
pod/pv-pod created
root@Master:~/k8s-08# kubectl get pod pv-pod
NAME      READY   STATUS    RESTARTS   AGE
pv-pod    1/1     Running   0           11s
root@Master:~/k8s-08#
```

- Test the PV/PVC - Create a file inside the pod:

```
root@Master:~/k8s-08# kubectl exec -it pv-pod -- sh
# echo "PV PVC Working" > /usr/share/nginx/html/test.txt
# exit
```

- Now exit and check outside output

```
root@Master:~/k8s-08# cat /data/pv/test.txt
PV PVC Working
root@Master:~/k8s-08#
```

#### 4. Set Up an Amazon EFS Volume and Attach it to Multiple Pods

- Create shared-pod1.yaml

```
root@Master:~/k8s-08# mkdir -p /data/shared
root@Master:~/k8s-08# vi shared-pod1.yaml
root@Master:~/k8s-08#
```

```

apiVersion: v1
kind: Pod
metadata:
  name: shared-pod1
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: shared-storage
      mountPath: /shared

  volumes:
  - name: shared-storage
    hostPath:
      path: /data/shared
      type: DirectoryOrCreate

```

- Apply shared-pod1.yaml file

```

root@Master:~/k8s-08# kubectl apply -f shared-pod1.yaml
pod/shared-pod1 created
root@Master:~/k8s-08#

```

- Create shared-pod2.yaml file

```

root@Master:~/k8s-08# vi shared-pod2.yaml
root@Master:~/k8s-08#

```

```

apiVersion: v1
kind: Pod
metadata:
  name: shared-pod2
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: shared-storage
      mountPath: /shared

  volumes:
  - name: shared-storage
    hostPath:
      path: /data/shared
      type: DirectoryOrCreate

```

- Apply shared-pod2.yaml file

```

root@Master:~/k8s-08# kubectl apply -f shared-pod2.yaml
pod/shared-pod2 created
root@Master:~/k8s-08#

```

- Verify both pods

```

root@Master:~/k8s-08# kubectl get pods

```

| NAME                                 | READY | STATUS  | RESTARTS | AGE   |
|--------------------------------------|-------|---------|----------|-------|
| nginx-deployment-f9db776c5-qfcvj     | 1/1   | Running | 0        | 5m28s |
| recreate-deployment-7c6d446b89-tps2m | 1/1   | Running | 0        | 5m28s |
| shared-pod1                          | 1/1   | Running | 0        | 3m22s |
| shared-pod2                          | 1/1   | Running | 0        | 54s   |

- Write a file from Pod 1 and exit

```
root@Master:~/k8s-08# kubectl exec -it shared-pod1 -- sh
# echo "Shared Storage Works" > /shared/demo.txt
# exit
```

- Read the same file from Pod 2:

```
root@Master:~/k8s-08# kubectl exec shared-pod2 -- cat /shared/demo.txt
Shared Storage Works
root@Master:~/k8s-08#
```

## 5. Implement and Test Liveness and Readiness Probes in a Kubernetes Pod

- Create probe-pod. Yaml

```
root@Master:~/k8s-08# vi probe-pod.yaml
root@Master:~/k8s-08#
```

```
apiVersion: v1
kind: Pod
metadata:
  name: probe-pod
spec:
  containers:
  - name: nginx
    image: nginx

    ports:
    - containerPort: 80

    livenessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 10
      periodSeconds: 5

    readinessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 5
      periodSeconds: 5
```

- Apply and verify it

```
root@Master:~/k8s-08# kubectl apply -f probe-pod.yaml
pod/probe-pod created
root@Master:~/k8s-08# kubectl get pod probe-pod
NAME          READY   STATUS    RESTARTS   AGE
probe-pod     1/1     Running   0           11s
root@Master:~/k8s-08#
```

- Verify pods

```
root@Master:~/k8s-08# kubectl describe pod probe-pod | grep -A10 "Liveness"
  Liveness:      http-get http://:80/ delay=10s timeout=1s period=5s #success=1 #failure=3
  Readiness:     http-get http://:80/ delay=5s timeout=1s period=5s #success=1 #failure=3
  Environment:   <none>
  Mounts:
    /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-wk9gg (ro)
  Conditions:
```

- Simulate a liveness probe failure
- Delete the default NGINX page inside the container:

```
root@Master:~/k8s-08# kubectl exec -it probe-pod -- rm /usr/share/nginx/html/index.html
root@Master:~/k8s-08# kubectl get pod probe-pod -w
NAME          READY   STATUS    RESTARTS   AGE
probe-pod     1/1     Running   0           2m18s
probe-pod     0/1     Running   0           2m20s
probe-pod     0/1     Running   1 (1s ago) 2m22s
probe-pod     1/1     Running   1 (9s ago) 2m30s
```

- Now inspect it:

```
root@Master:~/k8s-08# kubectl describe pod probe-pod
Name:          probe-pod
Namespace:     default
Priority:       0
Service Account: default
Node:          master/172.31.37.70
Start Time:    Sun, 05 Jul 2026 07:42:44 +0000
Labels:        <none>
Annotations:   cni.projectcalico.org/containerID: 3249211f2a3907dea37e80adf8e86a26b3791af24af9
               cni.projectcalico.org/podIP: 192.168.219.126/32
               cni.projectcalico.org/podIPs: 192.168.219.126/32
Status:        Running
IP:            192.168.219.126
IPs:
Events:
  Type     Reason      Age      From          Message
  ----     -
  Normal   Scheduled   2m52s    default-scheduler   Successfully assigned default/probe-pod to mast
  Normal   Pulled      2m52s    kubelet         Successfully pulled image "nginx" in 141ms (141
  Warning  Unhealthy   33s (x3 over 43s)  kubelet         Liveness probe failed: HTTP probe failed with s
  Normal   Pulling     32s (x2 over 2m52s)  kubelet         Pulling image "nginx"
  Normal   Created     32s (x2 over 2m52s)  kubelet         Created container: nginx
  Normal   Started     32s (x2 over 2m52s)  kubelet         Started container nginx
  Warning  Unhealthy   32s (x5 over 43s)  kubelet         Readiness probe failed: HTTP probe failed with
  Normal   Killing     32s          kubelet         Container nginx failed liveness probe, will be
  Normal   Pulled      32s          kubelet         Successfully pulled image "nginx" in 136ms (136
```